

03. 모델링 및 소스 코드

키 스트로크 특징값 정제 · 파생 · 스케일



통계적 검증 및 Pilot Modeling



데이터 확보



모델 학습

1

index.html

웹 화면 (편지)

타이핑 기록 →
기록을 서버로 전송

computeFeatures()

2

app.py

Streamlit 서버

타이핑 기록 받기
결과 화면 표시

letter_result.get('features')

3

logic.py

파이썬 예측 엔진

타이핑 요약 + 확률 →
최종 점수 산출

summarize_strokes()
predict_parkinson()

4

model.pkl

로지스틱 (3개 특징)

학습된 모델 파일
binary classifier

train_model.py 산출물

★ **app.py** 는 타이핑 기록을 받는 곳이자, 최종 점수를 화면에 보여주는 곳

타이핑을 있는 그대로 기록 (누를 때 / 땔 때)



데이터 확보

```
ta.addEventListener("input", refresh);

ta.addEventListener("keydown", function(ev){
  const t = performance.now();
  if (ev.code === "Backspace") { backspaceCount++; downSeq.push(t); drawWaveform(); return; }
  if (NON_PRINT.has(ev.code)) return;
  if (!(ev.code in pending)) { pending[ev.code] = { t: t, key: ev.key }; }
  downSeq.push(t);
  drawWaveform();
});

ta.addEventListener("keyup", function(ev){
  const t = performance.now();
  if (ev.code in pending) {
    strokes.push({ down: pending[ev.code].t, up: t, key: pending[ev.code].key });
    delete pending[ev.code];
  }
});
```

① performance.now()

- ✓ 브라우저의 정밀 시계(밀리초 단위)로
'누른 시각'을 기록합니다.
- ✓ 일반 시계보다 정밀해 타이핑 리듬의
미세한 차이를 잡아냅니다.

② 누름·땔 짝짓기

- ✓ 누른 시각을 **잠시 저장**해 두었다가, 키를 떼면
{누름, 땔, 키} **한 쌍**으로 확정합니다.
- ✓ 모든 **특징**의 재료가 됩니다.

타이핑 기록만 전송



데이터 확보

```
function computeFeatures() {
  const done = strokes.slice().sort((a,b)=>a.down-b.down);

  map(s => ({ h: (s.up - s.down)/1000, k: (s.key||"").toLowerCase() })))
  .filter(o => o.h >= 0 && o.h < 3);
  const holds_s = raw_strokes.map(o => o.h);
  const features = raw_strokes;

  //rhythm heuristic (computeLetterMetrics, ms)
  const intervals_ms = [];
  for (let i=1;i<downSeq.length;i++){
    const d = downSeq[i]-downSeq[i-1];
    if (d>0 && d<6000) intervals_ms.push(d);
  }
}
```

```
function refresh() {
  cnt.textContent = ta.value.length + "자";
  send.disabled = ta.value.trim().length < 80;
}
```

```
send.addEventListener("click", function(){
  if (ta.value.trim().length < 80) return;
});
```

① function computeFeatures()

브라우저는 특징을 직접 계산하지 않습니다.

타이핑 기록 (누른 시간+키)만 넘기고, 요약은 logic.py가 학습 때와 동일한 코드로 처리 → 학습과 실사용이 완전히 동일.

② 누름·땀 짝짓기

✓ 글자가 너무 적으면 특징이 흔들립니다.

✓ 80자 정도면 왼손 키가 약 35개 → 오차 약 12%로 안정됩니다.

타이핑 기록 요약 → 확률 구하기 & 점수화 → 상태 나누기



데이터 확보

```
def summarize_strokes(strokes):
    """브라우저가 넘긴 raw 스트로크 리스트를 학습과 동일하게 요약한다.
    strokes: [{"h": hold_seconds, "k": key_char}, ...]
    반환: (features_dict, 유효_타수)
    """
    holds, left = [], []
    for st in strokes or []:
        try:
            h = float(st.get("h"))
        except (TypeError, ValueError):
            continue
        if not (HOLD_LO <= h <= HOLD_HI):
            continue
        holds.append(h)
        if str(st.get("k", "")).lower() in LEFT_KEYS:
            left.append(h)
    hold_std = float(np.std(holds, ddof=1)) if len(holds) > 1 else 0.0
    hold_p90 = float(np.percentile(holds, 90)) if holds else 0.0
    left_mean = float(np.mean(left)) if left else 0.0
    left_cv = float(np.std(left, ddof=1) / left_mean) if len(left) > 1 and left_mean != 0 else 0.0
    return {"hold_std": hold_std, "left_cv": left_cv, "hold_p90": hold_p90, len(holds)}
```

① 타이핑 기록 요약

학습 코드(train_model.py)와 동일한 규칙으로 **3개 특징**을 계산 → 학습과 실사용 일치.

② 확률 구하기 & 점수화

모델은 '**위험 확률**'을 내놓습니다 → 사용자에게 '건강 점수'로: $100 \times (1 - \text{확률})$.

③ 상태 나누기

임의로 정한 값이 아니라, 학습으로 구한 **66.6/49.9**를 불러와 사용.

타이핑 기록 요약 → 확률 구하기 & 점수화 → 상태 나누기

```
prob = parkinson_model.predict_proba(df_input)[0][risk_idx]
final_score = int(round(100 * (1 - prob)))
final_score = max(0, min(100, final_score))
```

```
def band_for(score):
    """
    안정도 점수(=100x(1-파킨슨확률))를 3구간으로 분류한다.
    상한 upper(안정 경계, 민감도 85%) / 하한 lower(상담 경계, 특이도 85%)는
    ml_score_thresholds.json 에서 읽는다. 파일이 없으면 66.6 / 49.9 를 사용한다.
    """
    if ml_thresholds is not None and "upper" in ml_thresholds and "lower" in ml_thresholds:
        upper = ml_thresholds["upper"] # 안정 경계 (민감도 기준)
        lower = ml_thresholds["lower"] # 상담 경계 (특이도 기준)

        if score >= upper:
            return {"label": "안정", "cls": "safe"}
        if score <= lower:
            return {"label": "상담 권장", "cls": "alert"}
        return {"label": "관찰 필요", "cls": "watch"}

    if score >= 66.6:
        return {"label": "안정", "cls": "safe"}
    if score > 49.9:
        return {"label": "관찰 필요", "cls": "watch"}
    return {"label": "상담 권장", "cls": "alert"}
```

① 타이핑 기록 요약

학습 코드(train_model.py)와 동일한 규칙으로 3개 특징을 계산 → 학습과 실사용 일치.

데이터 확보

② 확률 구하기 & 점수화

모델은 '위험 확률'을 내놓습니다 → 사용자에게 '건강 점수'로: $100 \times (1 - \text{확률})$.

③ 상태 나누기

임의로 정한 값이 아니라, 학습으로 구한 66.6/49.9를 불러와 사용.

로지스틱 회귀 · 타이핑 3특징으로 건강 점수 산출

모델 구성 · 파라미터

3개 특징

hold_std, 기여도 0.81 · 정상↔PDp<0.001
left_cv, 기여도 0.99 · 가장 큰 신호
hold_p90, 기여도 0.70 · 정상↔PDp<0.001

누름시간 흔들림 (표준편차)
 왼손 타이핑 변동 (변동계수)
 느린 구간 상위 10%

C

1.0 (L2)

정규화 세기 — 조정해도 기본값이 최적

전처리

(중앙값 대체)

median imputer → StandardScaler

빈 값 채우기 + 표준화

random_state 42

재현성 고정

Scikit-learn Pipeline 코드

```
model = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(C=1.0, max_iter=5000, random_state=42)),
])
```



모델 학습

왜 로지스틱인가

1 6개 모델 중 최고
 AUC 0.860 · 비교군 대비 상위 성능

2 적은 인원(85명)에 강함
 RF 0.75 · SVM 0.85대비 우수

3 각 특징의 기여도 해석 가능
 계수(coefficient)로 각 특징이
 결과에 얼마나 기여했는지 파악

4 조정 불필요 · 기본값 최적
 어떤 하이퍼파라미터로도 ≈0.86 — 안정적

SHAP으로 본 피쳐 중요도

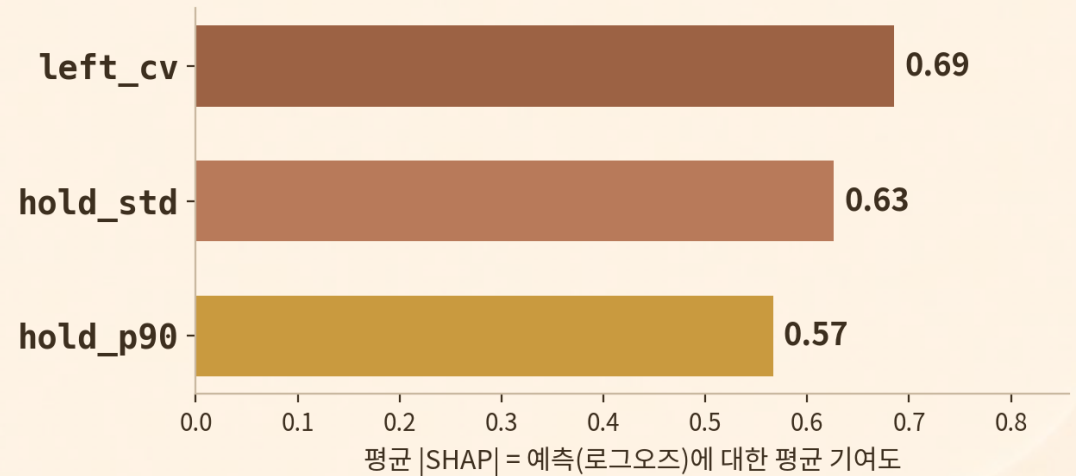


모델 학습

SHAP이란, 그리고 왜 계수로 충분한가

- SHAP = 각 특징이 예측에 얼마나 기여했는지
공정하게 분해해 보여주는 해석 기법
- 로지스틱은 예측 = $\Sigma(\text{계수} \times \text{특징값})$
→ 계수 자체가 곧 각 특징의 기여도
- 세 특징을 표준화(Z-score)했기 때문에
→ 계수를 직접 비교해 중요도로 해석 가능
- SHAP은 계수가 없는 복잡한 모델
(RF·딥러닝)을 근사 설명할 때 주로 사용

로지스틱 3특징의 평균 |SHAP| 기여도



left_cv (왼손 변동)가 가장 큰 기여 → 계수 순서(0.99 > 0.81 > 0.70)와 정확히 일치



로지스틱은 계수가 곧 SHAP 기여도라 별도 SHAP 없이도 해석 가능
세 특징 모두 위험을 높이는 방향, 왼손 변동이 최대 신호(좌우 비대칭이라는 임상 근거와 일치).

어떻게 검증했는가 — 교차 확인

0.860

AUC

0.800

ACCURACY

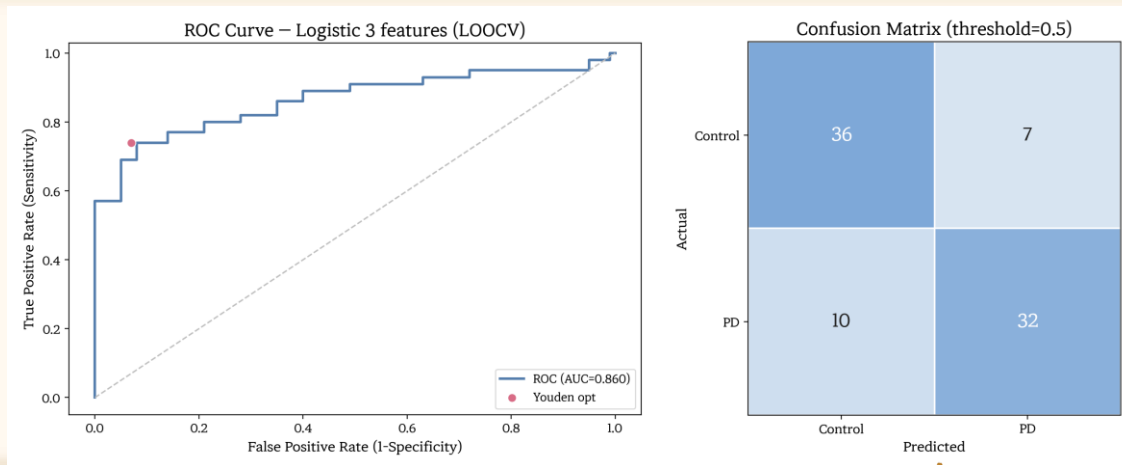
0.762

SENSITIVITY

0.837

SPECIFICITY

ROC Curve & Confusion Matrix (threshold = 0.5)



검증

```
# LOOCV 성능
proba_cv = cross_val_predict(model, X, y, cv=LeaveOneOut(), method="predict_proba")[:, 1]
pred_cv = (proba_cv >= 0.5).astype(int)
tn, fp, fn, tp = confusion_matrix(yb, pred_cv).ravel()
```

교차검증 4종

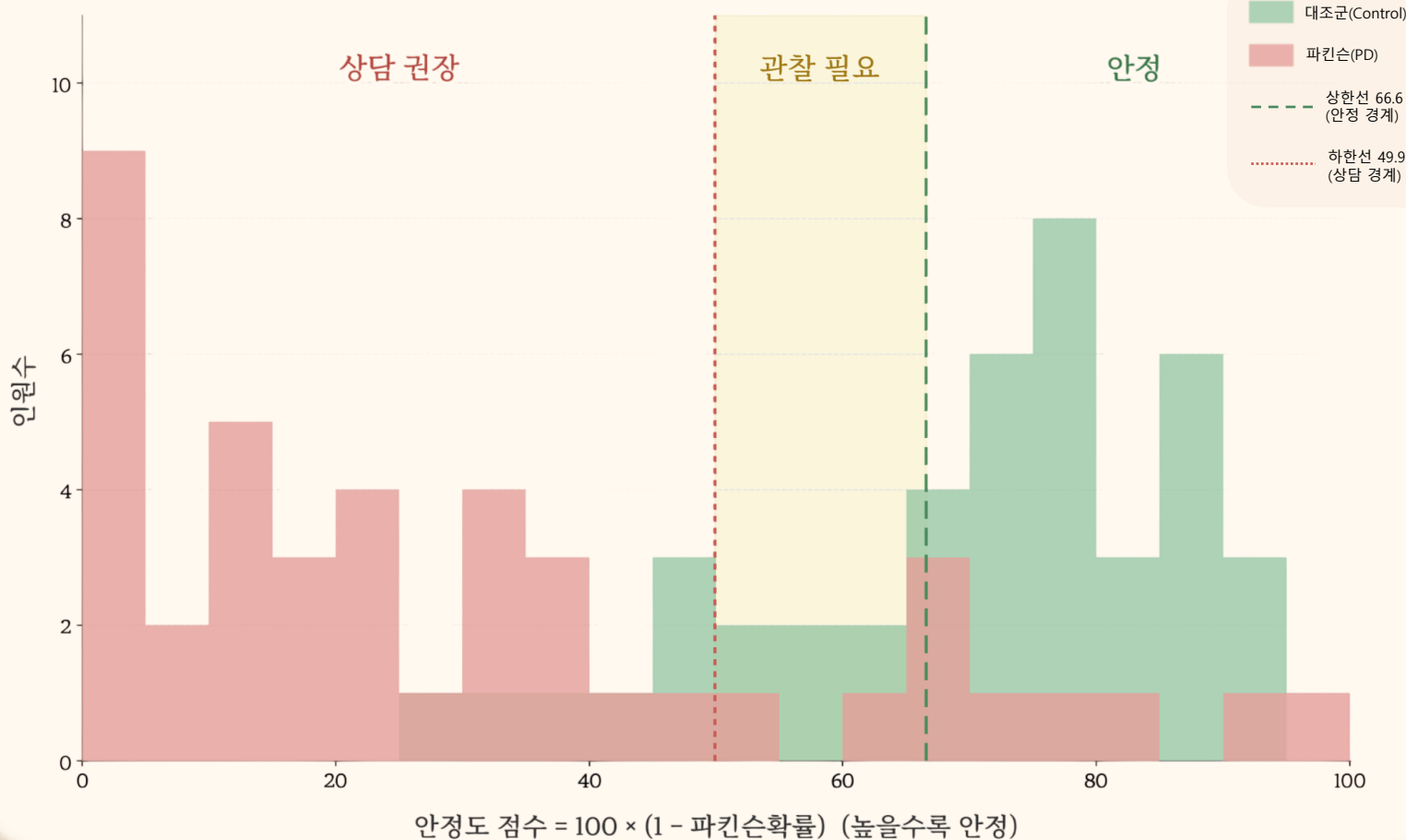
LOOCV	0.860
5-Fold	0.852
10-Fold	0.862
Nested CV	0.853

★ KEY FINDING

LOOCV(0.860) ↔ Nested CV(0.853) 차이 0.007
→ 성능 부풀림 없이 견고, 85명이라 한 명씩 빼고 84명으로 학습·검증을 반복해 데이터를 최대한 활용

점수는 이렇게 나옵니다 - 안정 관찰 상담 (3단계)

점수 분포와 3구간 컷오프



검증

점수 3단계

- **안정 · 66.6점 이상**
정상에 가까움
- **관찰 필요 · 49.9~66.6점**
애매하면 다시 확인
- **상담 권장 · 49.9점 이하**
신호가 뚜렷 → 상담

기준선 근거

- 66.6 = 민감도 85% (환자 놓침 최소)
- 49.9 = 특이도 85% (정상 오탐 최소)